2.

Let n be the number waiting (0 to 200), and let a = 0 mean wait and a = 1 mean dispatch. Dispatch happens before holding costs and new arrivals. One shuttle can be sent each period.

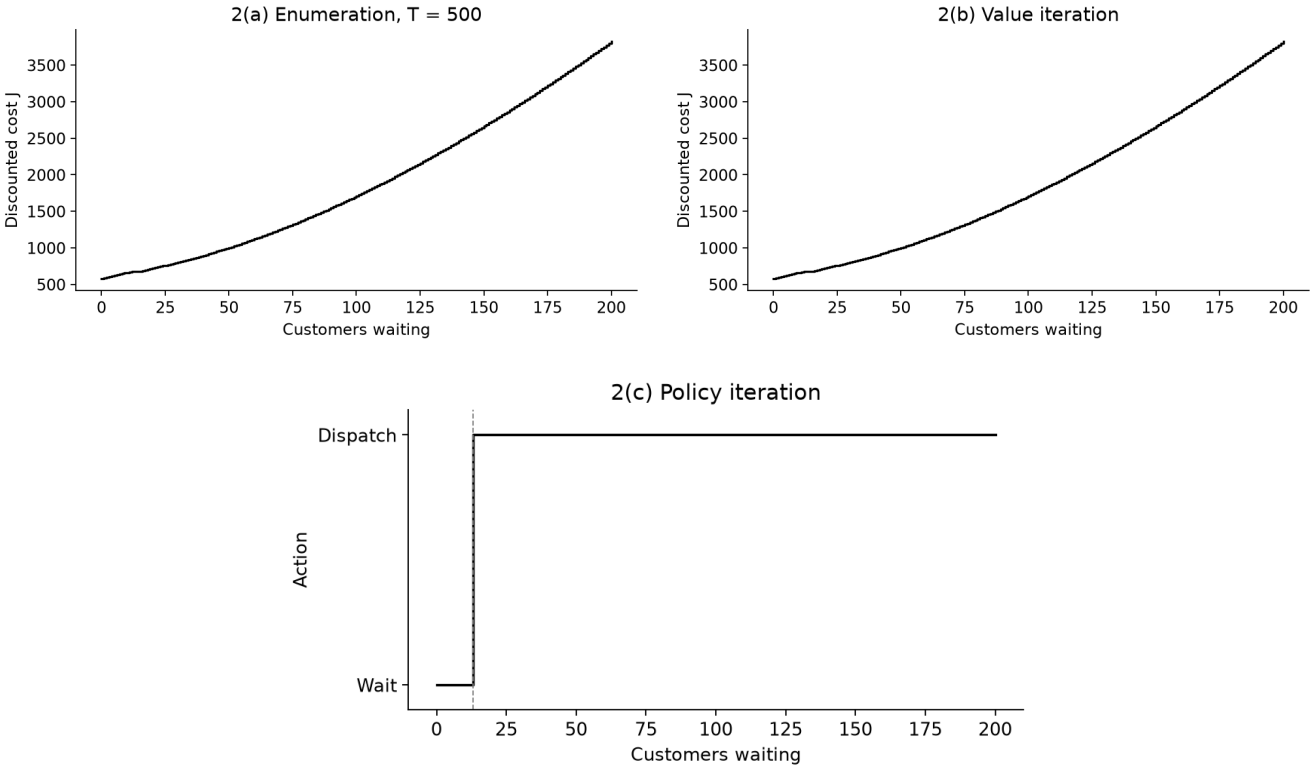
```
q = max(n - 15a, 0);  c(n,a) = 100a + 2q
n_next = min(200, q + A),  A uniform on {1,2,3,4,5}
J_t(n) = min_a [c(n,a) + 0.95 E J_(t+1)(n_next)]
```

J is discounted cost; reward value is V = -J. Excess arrivals are blocked without an extra penalty. For T=500, the terminal cost is zero.

2(a). Enumerate all states, actions and arrivals by backward induction for 500 periods.

2(b). Start value iteration at zero and stop when the largest change is below 1e-10.

2(c). Alternate exact policy evaluation, (I - gamma P)J = c, and greedy improvement.



Result: dispatch when n is at least 13; otherwise wait. J(0)=576.670, J(15)=676.670, and J(200)=3821.586. VI and PI give the same policy, with maximum cost difference 1.87e-9. The T=500 and infinite-horizon values differ by at most 4.94e-9.

Edge cases and checks: max(n-15,0) handles partial loads. At n=200, waiting keeps the queue at 200; dispatching gives next states 186-190. Empty queues and ties choose wait. With gamma=0, dispatch costs 100 but saves at most 30 immediately, so all states wait and J(n)=2n exactly. Free dispatch serves every nonempty queue; free holding gives J=0 and waiting everywhere. Both match the computed results.

3. Part 1

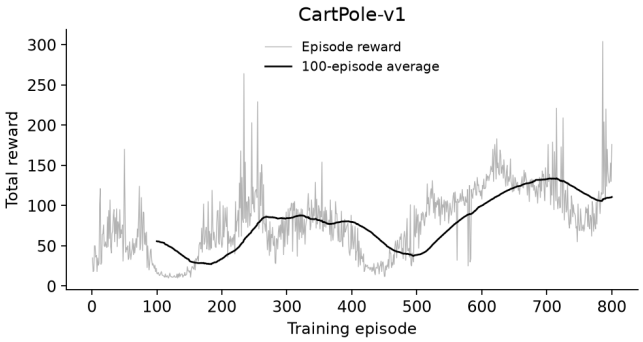
I use PyTorch REINFORCE with one Adam update after each episode. The objective is expected discounted episode reward. Actions are sampled from a two-output policy network.

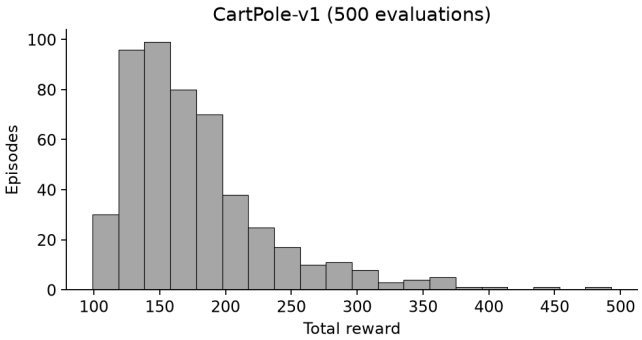
```
G_t = sum_(k=t)^(T-1) gamma^(k-t) r_k
Loss = -sum_t gamma^t log pi(a_t|s_t) G_t
```

CartPole: 4 inputs, 32 tanh units, 2 outputs; gamma=0.95, learning rate=0.01, 800 episodes. Pong: 6400 inputs, 32 ReLU units, 2 outputs; gamma=0.99, learning rate=0.001 (a conservative stability choice). I first tried 250 episodes, then reran training for 1000 total episodes with the same seed and settings. The first 250 episodes matched the initial run.

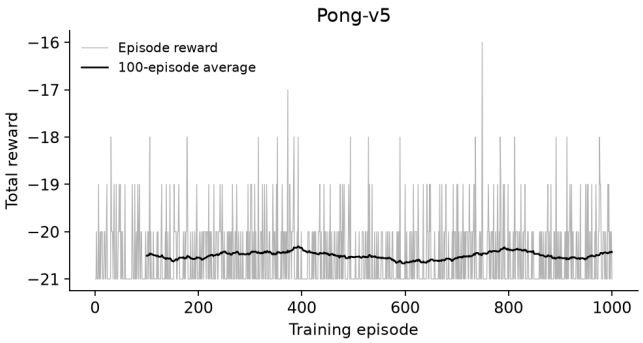
For ALE/Pong-v5, only actions 2 and 3 are used. I apply the supplied 80x80 binary preprocessing, then subtract the previous frame to provide motion information. The first difference is zero. Frame skip is 4 and sticky-action probability is 0.25.

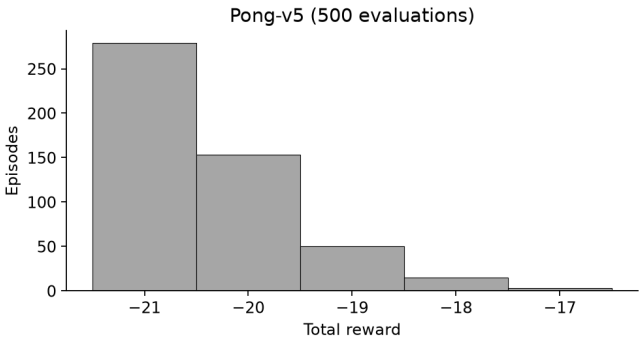
3(a).





3(b). 1000-episode run.





Environment / training episodes	Eval mean	Eval SD
CartPole-v1 / 800	176.82	55.99
Pong-v5 / 250	-20.50	0.71
Pong-v5 / 1000	-20.38	0.83

Plots show total reward and a trailing 100-episode mean. Each model, including both Pong attempts, is evaluated for 500 full episodes with a frozen, stochastic policy. SD uses denominator 500. Training seed: 7; evaluation seeds: 20000-20499.

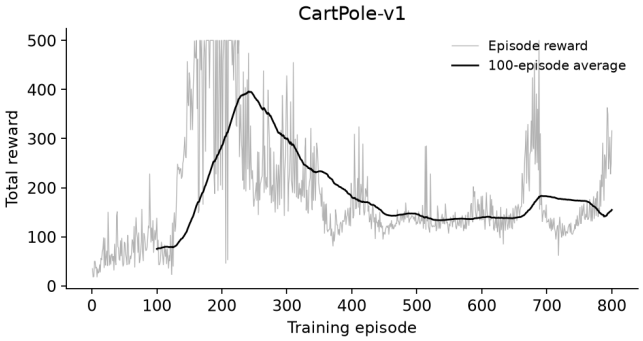
Checks and edge cases: 50 random-policy episodes give means 20.16 for CartPole and -20.58 for Pong. The 1000-episode Pong run improves by only 0.12 points and still loses all 500 evaluations. Termination and time-limit truncation end episodes; terminal rewards are kept. Returns and frame history reset between episodes, not between Pong points. No return normalization or reward shaping is used.

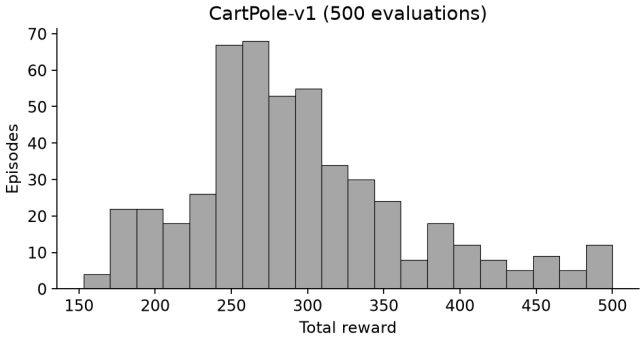
Policy Gradient: Part 2

I repeat both experiments with an exponential moving average (EMA) baseline. It starts at zero and stays fixed for the current episode. After the policy update, b is replaced by 0.9b + 0.1 mean(G_t). Thus the baseline uses previous episodes and does not depend on the current action. The baseline comparison uses the original budgets: 800 CartPole and 250 Pong episodes, with the other settings unchanged.

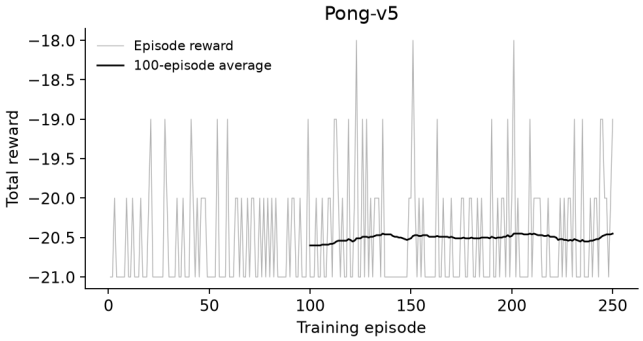
```
Loss = -sum_t gamma^t log pi(a_t|s_t) (G_t - b)
```

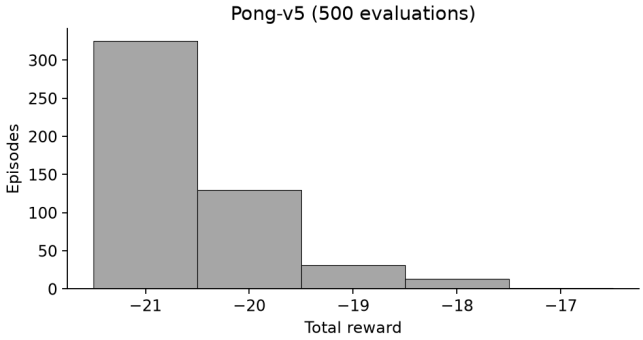
(a).





(b).





Environment / training episodes	Eval mean	Eval SD
CartPole-v1 / 800	294.39	72.46
Pong-v5 / 250	-20.53	0.74

The baseline raises CartPole mean reward from 176.82 to 294.39 in this run. Pong stays near -20.5, so it is not solved. One seed and a short training budget are not enough to conclude that one method is always better.

Variance check: for each final baseline-trained policy, I reuse its first 20 evaluation episodes and compute gradients with b=0 and with the final EMA. The sample covariance trace falls from 7823.25 to 50.41 for CartPole and from 418.84 to 116.25 for Pong. Lower gradient variance does not necessarily mean higher game scores.

Edge cases: the first episode uses b=0. The EMA is updated only after the optimizer step, avoiding a baseline fitted to the current episode. It works with zero or negative rewards without dividing by a reward standard deviation. A constant baseline is unbiased, but a poor choice can increase variance.

2. Validation: I also checked a small deterministic problem: capacity 3, cap 8, arrivals 2, dispatch cost 5, holding cost 1, gamma=0.8 and horizon 4. Enumerating all 16 action sequences from each of 9 states agrees with dynamic programming to 3.55e-15. With no arrivals and n=0,...,15, the analytic cost min(40n,100) matches the algorithm to 1.89e-9.

Policy Gradient, Parts 1 and 2. Validation: rewards [1,2,3] with gamma=0.5 give returns [2.75,3.5,3], matching hand calculation. In a one-step bandit with P(a=1)=0.3 and reward=a, exact enumeration gives gradient 0.21 with or without baseline 0.3. Its variance falls from 0.1029 to 0.0336 (67.3%). This tests the baseline idea independently of game performance. Evaluation leaves all model parameters unchanged.

Reproduce: use Python 3.12 and run the commands below from the repository root. Create a virtual environment and install requirements.txt first.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python shuttle.py
python policy_gradient.py --task cartpole --episodes 800 \
  --learning-rate 0.01 --output results/cartpole_plain
python policy_gradient.py --task cartpole --baseline --episodes 800 \
  --learning-rate 0.01 --output results/cartpole_baseline
python policy_gradient.py --task pong --episodes 250 \
  --learning-rate 0.001 --output results/pong_plain
python policy_gradient.py --task pong --episodes 1000 --learning-rate 0.001 \
  --output results/pong_plain_1000
python policy_gradient.py --task pong --baseline --episodes 250 \
  --learning-rate 0.001 --output results/pong_baseline
python policy_gradient.py --validate --output results/policy_validation.json
python -m pytest -q
python build_report.py
```

shuttle.py runs the parameter checks and exhaustive comparison. The --validate command runs the bandit, return and random-policy checks. Each training command also runs 500 evaluations and the 20-episode gradient check. Raw scores, metrics, first-10-step traces, models and plots are saved in results/.

With three more months: I would train Pong longer, repeat both methods over several seeds, and compare a learned value baseline with the constant EMA. I would test learning rates and episode batch sizes, measure gradient variance with more samples, and check shuttle policies under different queue caps and arrival distributions.

GitHub username, repository and link: ZikunZheng1129 / mlds490-hw1 / https://github.com/ZikunZheng1129/mlds490-hw1